NYC TAXI ANALYSIS SYSTEM

DOCUMENTACIÓN TÉCNICA

1 DE MARZO DE 2026

AWS DATA ENGINEERING

Proyecto Final - Academia Xideral

Contenido

1. RESUMEN EJECUTIVO 3
   1.1 DESCRIPCIÓN DEL PROYECTO 3
   1.2 OBJETIVOS CUMPLIDOS 3
   1.3 TECNOLOGÍAS UTILIZADAS 3
   1.4 DATOS PROCESADOS 4
   1.5 DESGLOSE POR AÑO 4
2. ARQUITECTURA DEL SISTEMA 5
   2.1 VISIÓN GENERAL 5
   2.2 FLUJO DE DATOS 6
   2.3 ESTRUCTURA DE ALMACENAMIENTO EN S3 7
3. COMPONENTES TÉCNICOS 8
   3.1 COMPONENTE: AWS LAMBDA (EXTRACCIÓN) 8
   3.2 COMPONENTE: EC2 (PROCESAMIENTO Y VISUALIZACIÓN) 9
   3.3 COMPONENTE: LIMPIEZA DE DATOS (PYSPARK) 10
   3.4 COMPONENTE: ANÁLISIS DE DATOS (PANDAS) 11
   3.5 COMPONENTE: DASHBOARD (STREAMLIT) 12
4. ESPECIFICACIONES TÉCNICAS DETALLADAS 12
   4.1 DEPENDENCIAS PYTHON 12
   4.2 CONFIGURACIÓN DE HADOOP/SPARK 13
   4.3 PERMISOS IAM 13
   4.4 COSTOS ESTIMADOS (MENSUALES) 14
5. RESULTADOS Y MÉTRICAS DEL PROYECTO 14
   5.1 MÉTRICAS GLOBALES (2021-2025) 14
   5.2 TENDENCIAS IDENTIFICADAS 15
   5.3 CALIDAD DE DATOS 15
   5.4 DESEMPEÑO DEL SISTEMA 15
6. MANUAL DE OPERACIÓN 16
   6.1 VERIFICACIÓN DEL SISTEMA 16
   6.2 INICIAR EL DASHBOARD 17
   6.3 ACCEDER AL DASHBOARD 17
   6.4 DETENER EL DASHBOARD 17
   6.5 RESOLUCIÓN DE PROBLEMAS 18

6.6 COMANDOS DE MANTENIMIENTO 19
CONCLUSIÓN 20
INFORMACIÓN DEL DOCUMENTO 20

1. RESUMEN EJECUTIVO

1.1 DESCRIPCIÓN DEL PROYECTO

El NYC Taxi Analysis System es un proyecto de análisis de datos masivos que procesa más de 1.1 mil millones de registros de viajes de taxi en Nueva York, utilizando servicios de AWS para implementar un pipeline ETL completo.

El sistema extrae datos desde la fuente pública de NYC TLC (Taxi & Limousine Commission), los procesa usando PySpark para limpieza y transformación, genera análisis estadísticos con Pandas, y presenta los resultados a través de un dashboard interactivo construido con Streamlit.

1.2 OBJETIVOS CUMPLIDOS

- Pipeline ETL completo implementado y funcional
- Procesamiento de 1.1 mil millones de registros (5 años de datos)
- Análisis estadístico con múltiples dimensiones temporales
- Dashboard interactivo con 5 páginas de visualización
- Calidad de datos superior al 99.9%
- Arquitectura escalable en AWS

1.3 TECNOLOGÍAS UTILIZADAS

- AWS Lambda (Python 3.12) - Extracción de datos
- Amazon S3 - Almacenamiento de datos
- Amazon EC2 (t2.medium) - Procesamiento y visualización
- PySpark 3.5.8 - Limpieza de datos distribuida
- Pandas 2.1.4 - Análisis estadístico
- Streamlit 1.31.0 - Dashboard de visualización
- Plotly 5.18.0 - Gráficos interactivos

1.4 DATOS PROCESADOS

- Período analizado: 2021-2025 (5 años)
- Archivos procesados: 60 archivos Parquet
- Total de viajes: 1,101,794,543 (1.1 mil millones)
- Ingresos totales: $27,426,786,867 USD
- Tarifa promedio: $24.89
- Distancia promedio: 4.51 millas
- Calidad de datos: 99.9%

1.5 DESGLOSE POR AÑO

AÑO 2021:

- Viajes: 174,340,216
- Ingresos: $3,838,547,863
- Tarifa promedio: $22.02

AÑO 2022:

- Viajes: 212,136,461
- Ingresos: $5,029,748,581
- Tarifa promedio: $23.71

AÑO 2023:

- Viajes: 232,373,378
- Ingresos: $5,717,766,211
- Tarifa promedio: $24.60

AÑO 2024:

- Viajes: 239,408,342
- Ingresos: $6,277,564,287
- Tarifa promedio: $26.22

AÑO 2025:

- Viajes: 243,536,146
- Ingresos: $6,563,159,925
- Tarifa promedio: $26.95

2. ARQUITECTURA DEL SISTEMA

2.1 VISIÓN GENERAL

El sistema implementa una arquitectura de datos moderna en AWS, separada en cuatro capas principales:

1. CAPA DE EXTRACCIÓN

- AWS Lambda descarga archivos Parquet desde NYC TLC
- Almacenamiento en S3 (bucket: xideralaws-curso-edson)
- Validación de integridad de archivos

2. CAPA DE TRANSFORMACIÓN

- PySpark en EC2 para limpieza distribuida
- Validación de calidad de datos
- Generación de reportes de calidad (JSON)

3. CAPA DE ANÁLISIS

- Pandas en EC2 para agregaciones y métricas
- Análisis temporal (anual, mensual, horario, semanal)
- Generación de archivos JSON con resultados

4. CAPA DE VISUALIZACIÓN

- Dashboard Streamlit en EC2
- 5 páginas interactivas con gráficos Plotly
- Carga dinámica desde S3

2.2 FLUJO DE DATOS

```
[NYC TLC Data Source]
        |
        | (HTTPS Download)
        v
  [AWS Lambda Extract]
        |
        | (Upload)
        v
   [Amazon S3 - Raw Data]
   /nyc-taxi/raw-data/fhvhv/YYYY/MM.parquet
        |
        | (Read S3A)
        v
  [EC2 - PySpark Clean]
        |
        | (Write S3A)
        v
   [Amazon S3 - Cleaned Data]
   /nyc-taxi/processed-data/cleaned/
   /nyc-taxi/processed-data/quality-reports/
        |
        | (Read via boto3)
        v
  [EC2 - Pandas Analysis]
        |
        | (Write via boto3)
```

```
        v
    [Amazon S3 - Analysis Results]
    /nyc-taxi/analysis-results/by-year/
    /nyc-taxi/analysis-results/index.json
        |
        | (Load via boto3)
        v
  [EC2 - Streamlit Dashboard]
        |
        | (HTTP)
        v
    [Usuario - Navegador Web]
```

2.3 ESTRUCTURA DE ALMACENAMIENTO EN S3

```
xideralaws-curso-edson/
└── nyc-taxi/
    ├── raw-data/
    |   └── fhvhv/
    |       ├── 2021/ (12 archivos .parquet)
    |       ├── 2022/ (12 archivos .parquet)
    |       ├── 2023/ (12 archivos .parquet)
    |       ├── 2024/ (12 archivos .parquet)
    |       └── 2025/ (12 archivos .parquet)
    |
    ├── processed-data/
    |   ├── cleaned/
    |   |   └── fhvhv/ (120+ archivos particionados)
```

```
│   └── quality-reports/
│       └── fhvhv/ (60 archivos JSON)
│
└── analysis-results/
    ├── by-year/
    │   ├── 2021_analysis.json
    │   ├── 2022_analysis.json
    │   ├── 2023_analysis.json
    │   ├── 2024_analysis.json
    │   └── 2025_analysis.json
    └── index.json
```

3. COMPONENTES TÉCNICOS

3.1 COMPONENTE: AWS LAMBDA (EXTRACCIÓN)

- Nombre de función: xideral_nyctaxi_extract_edson
- Runtime: Python 3.12
- Memoria asignada: 128 MB
- Timeout: 5 minutos (300 segundos)
- Región: us-west-1

FUNCIONALIDAD:

- Descarga archivos Parquet desde NYC TLC CloudFront
- Valida tamaño mínimo (>1MB)
- Sube a S3 con retry logic (3 intentos)
- Retorna estado: ok/failed/not_found/skipped

URL BASE DE DATOS:

https://d37ci6vzurychx.cloudfront.net/trip-data/

FORMATO DE EVENTOS:


```
{
  "start_year": 2020,
  "end_year": 2020,
  "datasets": ["fhvhv"],
  "start": 5,
  "batch_size": 1,
  "force": false
}
```


PERMISOS IAM REQUERIDOS:

- s3:PutObject en xideralaws-curso-edson/nyc-taxi/raw-data/*
- logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents

3.2 COMPONENTE: EC2 (PROCESAMIENTO Y VISUALIZACIÓN)

- Tipo de instancia: t2.medium
- vCPUs: 2
- Memoria RAM: 4 GB
- Almacenamiento: 64 GB SSD
- Sistema operativo: Ubuntu / Amazon Linux 2
- Región: us-west-1

SOFTWARE INSTALADO:

- Java 11 (OpenJDK) - Requerido por Spark
- Hadoop 3.4.2 - Para acceso S3A
- Spark 3.5.8 (standalone) - Procesamiento distribuido
- Python 3.12 con venv - Entorno de ejecución
- AWS CLI - Gestión de recursos AWS

CONFIGURACIÓN DE RED:

- IP pública asignada (acceso al dashboard)
- Security Group configurado:
    - Puerto 22 (SSH) - Solo desde IPs autorizadas
    - Puerto 8501 (Streamlit) - Acceso público o restringido

3.3 COMPONENTE: LIMPIEZA DE DATOS (PYSPARK)

- Script: clean_data.py
- Ubicación en EC2: ~/nyc-taxi-project/
- Lenguaje: Python 3.12 + PySpark 3.5.8

CONFIGURACIÓN SPARK:

- Modo: Standalone (sin cluster)
- Driver memory: 2GB
- Executor memory: 2GB
- S3A FileSystem habilitado

PROCESO DE LIMPIEZA:

1. Lectura de archivo Parquet desde S3
2. Eliminación de registros con nulos críticos:
    - pickup_datetime
    - dropoff_datetime
    - trip_miles
    - base_passenger_fare
3. Validación de fechas:
    - pickup_datetime < dropoff_datetime
    - Fechas dentro del rango del mes
4. Filtrado de outliers:
    - trip_miles: 0.1 - 100 millas
    - base_passenger_fare: $1 - $500
    - trip_time: 1 - 180 minutos
5. Normalización:
    - Flags booleanos a 0/1
    - Trim de strings
    - Redondeo de decimales

6. Generación de reporte de calidad (JSON)

MÉTRICAS DE RENDIMIENTO:

- Tiempo por archivo: 2-4 minutos
- Total 60 archivos: ~2.5-3 horas
- Calidad promedio: 99.9%

3.4 COMPONENTE: ANÁLISIS DE DATOS (PANDAS)

- Script: analyze_data.py
- Ubicación en EC2: ~/nyc-taxi-project/
- Lenguaje: Python 3.12 + Pandas 2.1.4

PROCESO DE ANÁLISIS:

1. Carga de datos limpios desde S3 (por año)
2. Generación de métricas agregadas:
    - total_trips
    - total_revenue
    - avg_fare
    - avg_distance
    - avg_duration
3. Cálculo de monthly_metrics (12 meses):
    - Agregación mensual
    - Identificación de tendencias
4. Análisis hourly_patterns (24 horas):
    - Viajes por hora del día
    - Tarifas promedio por hora
    - Identificación de horas pico
5. Análisis day_of_week_patterns (7 días):
    - Viajes por día de semana
    - Comparativa weekday vs weekend
6. Guardado de resultados en S3 (formato JSON)

MÉTRICAS DE RENDIMIENTO:

- Tiempo por año: 3-5 minutos

- Total 5 años: ~20-30 minutos

3.5 COMPONENTE: DASHBOARD (STREAMLIT)

- Framework: Streamlit 1.31.0
- Puerto: 8501
- Ubicación en EC2: ~/nyc-taxi-dashboard/

ESTRUCTURA DEL DASHBOARD:

- Home.py - Página principal con resumen global
- pages/1_Analisis_por_Año.py - Métricas anuales
- pages/2_Tendencias_Mensuales.py - Comparativa multi-año
- pages/3_Patrones_Horarios.py - Análisis por hora del día
- pages/4_Patrones_Semanales.py - Análisis por día de semana
- pages/5_Demo_en_Vivo.py - Análisis mensual individual

CARACTERÍSTICAS:

- Carga dinámica de datos desde S3
- Gráficos interactivos con Plotly
- Selectores de año y mes
- KPIs principales destacados
- Formato responsive

4. ESPECIFICACIONES TÉCNICAS DETALLADAS

4.1 DEPENDENCIAS PYTHON

PARA LIMPIEZA Y ANÁLISIS (EC2):

- pyspark==3.5.8
- pandas==2.1.4
- numpy==1.26.0
- boto3==1.42.68

- pyarrow==18.1.0

PARA DASHBOARD (EC2):

- streamlit==1.31.0
- plotly==5.18.0
- boto3==1.34.0
- pandas==2.1.4

4.2 CONFIGURACIÓN DE HADOOP/SPARK

JARs requeridos (auto-descarga):

- hadoop-aws-3.3.4.jar
- aws-java-sdk-bundle-1.12.262.jar
- wildfly-openssl-1.0.7.Final.jar

Configuración Spark S3A:

- spark.jars.packages: org.apache.hadoop:hadoop-aws:3.3.4
- spark.hadoop.fs.s3a.impl: org.apache.hadoop.fs.s3a.S3AFileSystem
- spark.hadoop.fs.s3a.aws.credentials.provider:
    - com.amazonaws.auth.DefaultAWSCredentialsProviderChain

4.3 PERMISOS IAM

LAMBDA EXECUTION ROLE:

- AWSLambdaBasicExecutionRole (managed policy)
- s3:PutObject en xideralaws-curso-edson/nyc-taxi/raw-data/*
- s3:GetObject (validación de archivos)

EC2 INSTANCE PROFILE:

- s3:GetObject en xideralaws-curso-edson/*
- s3:PutObject en xideralaws-curso-edson/nyc-taxi/processed-data/*

- s3:PutObject en xideralaws-curso-edson/nyc-taxi/analysis-results/*
- s3:ListBucket en xideralaws-curso-edson

4.4 COSTOS ESTIMADOS (MENSUALES)

- EC2 t2.medium (24/7): ~$34 USD
- S3 Storage (100 GB): ~$2-3 USD
- Lambda Invocations (<100): Gratis (Free Tier)
- Data Transfer: Mínimo (mismo region)
- TOTAL MENSUAL: ~$36-37 USD

OPTIMIZACIONES POSIBLES:

- Detener EC2 cuando no se use: -70%
- Reserved Instance EC2: -40%
- Lifecycle policies S3: -50% storage

5. RESULTADOS Y MÉTRICAS DEL PROYECTO

5.1 MÉTRICAS GLOBALES (2021-2025)

- Total de viajes procesados: 1,101,794,543
- Ingresos totales generados: $27,426,786,867
- Tarifa promedio: $24.89
- Distancia promedio: 4.51 millas
- Duración promedio: ~15 minutos
- Archivos raw procesados: 60
- Archivos limpios generados: 120+
- Reportes de calidad: 60
- Calidad de datos: 99.9%

5.2 TENDENCIAS IDENTIFICADAS

CRECIMIENTO ANUAL:

- Los viajes aumentaron un 39.7% entre 2021 y 2025
- Los ingresos crecieron un 71% en el mismo período
- La tarifa promedio subió $4.93 (22.4%)

PATRONES TEMPORALES:

- Horas pico: 8-10 AM y 5-7 PM (días laborables)
- Mayor demanda: Viernes y sábados
- Menor demanda: Domingos y lunes temprano

5.3 CALIDAD DE DATOS

- Registros originales: 1,102,900,000 (aprox)
- Registros después de limpieza: 1,101,794,543
- Tasa de retención: 99.9%

MOTIVOS DE DESCARTE:

- Nulos en campos críticos: <0.05%
- Fechas inválidas: <0.03%
- Outliers en valores: <0.02%

5.4 DESEMPEÑO DEL SISTEMA

FASE DE EXTRACCIÓN (Lambda):

- Tiempo por archivo: 30-90 segundos
- Total 60 archivos: ~60 minutos
- Tasa de éxito: 100%

FASE DE LIMPIEZA (PySpark):

- Tiempo por archivo: 2-4 minutos
- Total 60 archivos: 2.5-3 horas
- Throughput: ~7 millones registros/minuto

FASE DE ANÁLISIS (Pandas):

- Tiempo por año: 3-5 minutos
- Total 5 años: 20-30 minutos

DASHBOARD (Streamlit):

- Tiempo de carga inicial: 2-3 segundos
- Cambio de página: <1 segundo
- Actualización de gráficos: <0.5 segundos

6. MANUAL DE OPERACIÓN

6.1 VERIFICACIÓN DEL SISTEMA

VERIFICAR ESTADO DE STREAMLIT:

```
$ ps aux | grep streamlit
```

VERIFICAR PUERTO 8501:

```
$ sudo netstat -tulpn | grep 8501
$ sudo lsof -i :8501
```

VERIFICAR CREDENCIALES AWS:

```
$ aws configure list
$ aws s3 ls s3://xideralaws-curso-edson/nyc-taxi/
```

6.2 INICIAR EL DASHBOARD

PASO 1: Conectarse a EC2

$ ssh -i clave.pem ubuntu@<ip-publica-ec2>

PASO 2: Navegar al directorio

$ cd ~/nyc-taxi-dashboard

PASO 3: Activar entorno virtual

$ source ~/XideralAcademy/venv/bin/activate

PASO 4: Ejecutar Streamlit

$ streamlit run Home.py --server.port 8501 --server.address 0.0.0.0

MODO BACKGROUND (recomendado):

$ nohup streamlit run Home.py --server.port 8501 --server.address 0.0.0.0 > streamlit.log 2>&1 &

6.3 ACCEDER AL DASHBOARD

URL del dashboard:

http://<ip-publica-ec2>:8501

Verificar IP pública:

$ curl ifconfig.me

6.4 DETENER EL DASHBOARD

OPCIÓN 1: Por PID específico

```
$ sudo lsof -i :8501
$ kill <PID>
```

OPCIÓN 2: Detener todos los procesos Streamlit

```
$ pkill -f streamlit
```

6.5 RESOLUCIÓN DE PROBLEMAS

PROBLEMA: Puerto 8501 ocupado

SOLUCIÓN 1: Usar puerto alternativo

```
$ streamlit run Home.py --server.port 8502 --server.address 0.0.0.0
```

SOLUCIÓN 2: Liberar el puerto

```
$ sudo lsof -i :8501
$ kill <PID>
```

PROBLEMA: ModuleNotFoundError

SOLUCIÓN: Instalar dependencias

```
$ source ~/XideralAcademy/venv/bin/activate
$ pip install -r requirements.txt
```

PROBLEMA: AWS Credentials Error

SOLUCIÓN: Configurar credenciales

```
$ aws configure
  AWS Access Key ID: <tu-access-key>
  AWS Secret Access Key: <tu-secret-key>
  Default region: us-west-1
```

Default output format: json

PROBLEMA: Dashboard no carga datos

VERIFICAR:

- Conectividad a S3
- Permisos IAM del instance profile
- Existencia de archivos de análisis en S3

6.6 COMANDOS DE MANTENIMIENTO

VER LOGS DE STREAMLIT:

```
$ tail -f streamlit.log
```

VERIFICAR ESPACIO EN DISCO:

```
$ df -h
```

VERIFICAR MEMORIA:

```
$ free -h
```

LISTAR ARCHIVOS EN S3:

```
$ aws s3 ls s3://xideralaws-curso-edson/nyc-taxi/ --recursive
```

SINCRONIZAR DESDE S3 (si es necesario):

```
$ aws s3 sync s3://xideralaws-curso-edson/nyc-taxi/analysis-results/ ./local-backup/
```

CONCLUSIÓN

El NYC Taxi Analysis System demuestra la implementación exitosa de un pipeline ETL completo en AWS, procesando datos masivos (1.1 mil millones de registros) con una calidad superior al 99.9%.

El sistema combina servicios gestionados de AWS (Lambda, S3) con instancias EC2 para procesamiento distribuido (PySpark) y visualización interactiva (Streamlit), logrando un balance entre costo, rendimiento y escalabilidad.

Los resultados obtenidos proporcionan insights valiosos sobre los patrones de uso del servicio de taxi en Nueva York, identificando tendencias temporales, horarios pico y comportamientos estacionales que pueden ser utilizados para optimización operativa y toma de decisiones estratégicas.

INFORMACIÓN DEL DOCUMENTO

- Proyecto: NYC Taxi Analysis System
- Autor: Edson
- Institución: Academia Xideral
- Programa: AWS Data Engineering
- Fecha de generación: Marzo 2026
- Versión: 1.0 - Ejecutiva